Rapport sur : Projet FREECEL en langage C

Réalisé par : Boulghalegh Youssef

Etudiant en ENSAO . Filière : IDSCC3

I. Introduction :

Quand on regarde le jeu FRECEEL on voit qu'il se compose en 3 zones : zone 1, zone 2, et zone3 et chaque zone à ses propres propriétés mais le point commun entre eux sont les piles et les cartes de jeu. Donc d'abord la première chose c'est qu'on doit avoir toutes les piles (8 en zone1, 4 en zone 2, 4 en zone3) et toutes les cartes du jeu (52 cartes). Puis il faut remplir les piles de zone 1 par toutes les cartes aléatoirement (7 cartes dans les 4 premières piles et 6 cartes dans les 4 dernières piles). Ensuite il faut afficher tous cela à l'écran. Enfin, demander à chaque fois à l'utilisateur quelle opération il veut réaliser jusqu'à ce que le jeu termine soit par son gain ou bien sa perte.

II. Structures et fonction nécessaires

Quand on voit les cartes, on remarque qu'elles se composent de 3 attributs : le nombre, la couleur, et le type. Donc on doit avoir une structure pour cela :

```
typedef struct {
    char type; // HEART: H  diamonds : D   clubs : C    spades : S
    char couleur; // Rouge : R    Noir: N
    int chiffre;
}Card;
```

On doit avoir également une structure pour les piles qui vont contenir les cartes avec deux pointeurs (j'ai utilisé 2 pointeurs pour l'affichage car on doit afficher de la fin de la pile jusqu'à son début)

```
typedef struct element{
    Card *C;
    struct element *next;
    struct element *previous;
}Pile;
```

Et bien sûr on aura besoin des fonctions nécessaires pour manipuler les piles qui sont : **empiler, pop, peek**. Et la fonction **CountPile** qui va nous aider à calculer le nombre de cartes dans une pile.

```c
void empiler(Pile **P,Card *c){
    Pile *nouveau;
    nouveau=(Pile *)malloc(sizeof(Pile));
    nouveau->C=c;
    nouveau->previous=NULL;
    if(*P==NULL){
        nouveau->next=NULL;
        *P=nouveau;
        return;
    }
    nouveau->next=*P;
    (*P)->previous=nouveau;
    *P=nouveau;
}
```

```c
Card *peek(Pile *P){
    if(P==NULL) return;
    Card *c;
    c=P->C;
    return c;
}
```

```c
//countpile
int CountPile(Pile *P){
    int count=0;
    Pile *pt;
    for(pt=P;pt!=NULL;pt=pt->next){
        count++;
    }
    return count;
}
```

```c
void pop (Pile **P){
    if(*P==NULL) return;
    if (CountPile(*P) == 1){
        *P=NULL;
        return;
    }
    Pile *temp;
    temp = *P;
    *P=(*P)->next;
    (*P)->previous=NULL;
    free(temp);
}
```

III. 1ère étape :

Il faut avoir toutes les piles en déclarant trois tableaux de piles pour les 3 zones et initialiser chaque pile par NULL en utilisant la fonction '**PileNULL**'

```c
void PileNULL(Pile **P,int index){
    int i;
    for(i=0;i<index;i++){
        P[i]=NULL;
    }
}
int main() {
    //initialiser les piles
    Pile *Zone1[8]; //piles de zone1
    Pile*Zone2[4]; //piles de zone 2
    Pile*Zone3[4]; //piles de zone 3
    PileNULL(Zone1,8); PileNULL(Zone2,4); PileNULL(Zone3,4);
```

Puis il faut déclarer toutes les piles en utilisant la fonction ‘**initialiserCard**’

```
//initialiser les cartes
Card *ASH=initialiserCard(1,'H','R'); Card *ASD=initialiserCard(1,'D','R'); Card *ASC=initialiserCard(1,'C','N'); Card *ASS=initialiserCard(1,'S','N');
Card *_2H=initialiserCard(2,'H','R'); Card *_2D=initialiserCard(2,'D','R'); Card *_2C=initialiserCard(2,'C','N'); Card *_2S=initialiserCard(2,'S','N');
Card *_3H=initialiserCard(3,'H','R'); Card *_3D=initialiserCard(3,'D','R'); Card *_3C=initialiserCard(3,'C','N'); Card *_3S=initialiserCard(3,'S','N');
Card *_4H=initialiserCard(4,'H','R'); Card *_4D=initialiserCard(4,'D','R'); Card *_4C=initialiserCard(4,'C','N'); Card *_4S=initialiserCard(4,'S','N');
Card *_5H=initialiserCard(5,'H','R');   Card *_5D=initialiserCard(5,'D','R');   Card *_5C=initialiserCard(5,'C','N'); Card *_5S=initialiserCard(5,'S','N');
Card *_6H=initialiserCard(6,'H','R');   Card *_6D=initialiserCard(6,'D','R');   Card *_6C=initialiserCard(6,'C','N');   Card *_6S=initialiserCard(6,'S','N');
Card *_7H=initialiserCard(7,'H','R');   Card *_7D=initialiserCard(7,'D','R');   Card *_7C=initialiserCard(7,'C','N');   Card *_7S=initialiserCard(7,'S','N');
Card *_8H=initialiserCard(8,'H','R');   Card *_8D=initialiserCard(8,'D','R');   Card *_8C=initialiserCard(8,'C','N');   Card *_8S=initialiserCard(8,'S','N');
Card *_9H=initialiserCard(9,'H','R');   Card *_9D=initialiserCard(9,'D','R');   Card *_9C=initialiserCard(9,'C','N');   Card *_9S=initialiserCard(9,'S','N');
Card *_10H=initialiserCard(10,'H','R'); Card *_10D=initialiserCard(10,'D','R'); Card *_10C=initialiserCard(10,'C','N');Card *_10S=initialiserCard(10,'S','N')
Card *JH=initialiserCard(11,'H','R'); Card *JD=initialiserCard(11,'D','R'); Card *JC=initialiserCard(11,'C','N'); Card *JS=initialiserCard(11,'S','N');
Card *QH=initialiserCard(12,'H','R'); Card *QD=initialiserCard(12,'D','R'); Card *QC=initialiserCard(12,'C','N'); Card *QS=initialiserCard(12,'S','N');
Card *KH=initialiserCard(13,'H','R'); Card *KD=initialiserCard(13,'D','R'); Card *KC=initialiserCard(13,'C','N'); Card *KS=initialiserCard(13,'S','N');
```

```
Card *initialiserCard(int chiff,char ty , char coul){
    Card *C;
    C=(Card *)malloc(sizeof(Card));
    C->chiffre=chiff;
    C->type=ty;
    C->couleur=coul;
    return C;
}
```

IV. 2ème Etape :

Il faut empiler les piles de la zone 1 aléatoirement en utilisant la fonction melangerCartes qui va nous mélanger le tableau des cartes. Pour cela on aura besoin de la bibliothèque ‘**times.h**’ pour générer un nombre aléatoire qui va nous aider à mélanger le tableau des cartes ; puis empiler 7 cartes du tableau des cartes dans les 4 premières piles du zone1, et 6 cartes dans les 4 dernières piles du zone1.

```
void melangerCartes(Card *cartes[], int size) {
    int i;
    for (i = size - 1; i > 0; i--) {
        int j = rand() % (i + 1);
        // Échange les pointeurs de cartes[i] et cartes[j]
        Card *temp = cartes[i];
        cartes[i] = cartes[j];
        cartes[j] = temp;
    }
}
```

```
// Mélangez les cartes
int tailleCartes = sizeof(cartes) / sizeof(cartes[0]);
melangerCartes(cartes, tailleCartes);

//empiler les piles de la zone 1 aléatoirement
for (i = 0; i < 7; i++) {//empiler 7 cartes dans les 4 premières piles
    empiler(&Zone1[0], cartes[i]);
    empiler(&Zone1[1], cartes[i + 7]);
    empiler(&Zone1[2], cartes[i + 14]);
    empiler(&Zone1[3], cartes[i + 21]);
}
// Empilez 6 cartes dans les 4 dernières piles
for (i = 28; i < 34; i++) {
    empiler(&Zone1[4], cartes[i]);
    empiler(&Zone1[5], cartes[i + 6]);
    empiler(&Zone1[6], cartes[i + 12]);
    empiler(&Zone1[7], cartes[i + 18]);
}
```

V. 3ème étape :

Maintenant le jeu va commencer. Il nous faut d'abord d'afficher à l'écran la table du jeu avec les règles du jeu.

```
//afficher à l'écran
printf("---------------------------------------LES REGLES---------------------------------------------\n");
printf("|les piles de la zone 1 sont : A, B, C, D, E, F, G, H                                        | \n")
printf("|les piles de la zone 2 sont : I, J, K, L                                                    | \n")
printf("|les piles de la zone 3 sont : M(Clubs), N(Heart), O(Spades), P(Diamonds)                    | \n")
printf("|Pour deplacer une carte de la zone 1 ou 2 dans une pile de la zone 1 veuillez tapez sur 1 | \n")
printf("|Pour deplacer une carte de la zone 1 ou 2 dans une pile de la zone 2 veuillez tapez sur 2 | \n")
printf("|Pour deplacer une carte de la zone 1 ou 2 dans une pile de la zone 3 veuillez tapez sur 3 | \n")
printf("|Pour rejouez cette partie veuillez tapez sur 4                                              | \n")
printf("|Pour une nouvelle partie veuillez tapez sur 5                                               | \n")
printf("|Pour quitter veuillez tapez sur 6                                                           | \n")
printf("-----------------------------------------------------------------------------------------------\n");
Distribuer(Zone1,Zone2,Zone3);
```

L'affichage est en utilisant la fonction '**Distribuer'** :

```
void Distribuer(Pile **P1, Pile **P2, Pile **P3){
    printf("-------------------------------------------ZONE 1 : Zone des cartes------------------------------------------------------------ \n");
    Color(1,15);
    printf("\n A \t B \t C \t D \t E \t F \t G \t H \n");
    printf("\n");
    afficherPileZone1(P1);
    printf("\n---------------ZONE 2 : LES CELLULES LIBRES------------------------\n");
    printf("\nI : ");afficherPileZone2(P2[0]);
    printf("\tJ : ");afficherPileZone2(P2[1]);
    printf("\tK : ");afficherPileZone2(P2[2]);
    printf("\tL : ");afficherPileZone2(P2[3]);
    printf("\n\n------------------------ZONE 3 : LES CELLULES DE GAIN-----------------------------------\n");
    char H=3 , C=5 , D=4 , S=6 ;
    printf("CELLULE DE  CLUBS    %c   M : ",C);afficherPileZone3(P3[0]);
    printf("CELLULE DE HEART     %c   N : ",H);afficherPileZone3(P3[1]);
    printf("CELLULE DE SPADES    %C   O : ",S);afficherPileZone3(P3[2]);
    printf("CELLULE DE DIAMONDS  %C   P : ",D);afficherPileZone3(P3[3]);
}
```

Pour faciliter la communication avec l'utilisateur, la fonction **Distribuer** donne aux piles de la zone 1 un nom de 'A' à 'H', celles de la zone 2 de 'I' à 'L', celles de la zone 3 de 'M' à 'P' . La fonction se compose de 3 fonctions '**afficherZone1', 'afficherZone2',** '**afficherZone3**' car chaque zone à son affichage unique :

```c
void afficherPileZone1(Pile **P1){
    int i,j;
    Pile *pt[8];
    for(i=0;i<8;i++){
        pt[i]=P1[i];
    }
    Pile *temp[8];
    for(i=0;i<8;i++){
        temp[i]=arriverFin(pt[i]);
    }
    for(i=0;i<8;i++){
        pt[i]=temp[i];
    }
    while(pt[0]!=NULL || pt[1]!=NULL || pt[2]!=NULL || pt[3]!=NULL || pt[4]!=NULL || pt[5]!=NULL || pt[6]!=NULL || pt[7]!=NULL){
        for(i=0;i<8;i++){
            afficher(pt[i]);
            if(pt[i]!=NULL)
                pt[i]=pt[i]->previous;
        }
        printf("\n");
        printf("\n");
    }
}
```

Figure 1: Affichage des piles de la zone 1

```c
void afficherPileZone2(Pile *P){
    Pile *pt=P;
    afficher(pt);
}
    //pour afficher les piles de zone3
void afficherPileZone3(Pile *P){
    if(P==NULL){
        printf("\n");
        printf("\n");
        return;
    }
    Pile *pt;
    pt=P;
    Pile *temp=arriverFin(pt);
    for (pt = temp; pt != NULL; pt = pt->previous) {
        afficher(pt);
    }
    printf("\n");
    printf("\n");
}
```

Figure 2 : Affichage des piles de la zone 2 et la zone 3

Ces 3 fonctions utilisent les 2 fonctions ‘**arriverFin**’ et **‘afficher’**. **arriverFin** qui nous permet d’aller à la fin d’une liste pour qu’on puisse afficher de la fin ; et ‘**afficher’** qui nous permet d’afficher une seule carte.

```c
    //arriverfin : pour afficher du dernier element
Pile *arriverFin(Pile *pt){
    Pile *temp;
    if(pt==NULL) return NULL;
    while(pt!=NULL){
        temp=pt;
        pt=pt->next;
    }
    return temp;
}
```

```c
void afficher(Pile *pt){
    if(pt==NULL){
        printf("\t");
        return;
    }
    char type; //pour transformer le type qui est un caractere
        switch (pt->C->type) {
            case ('H'): type=3; break;  // Heart
            case ('S'): type=6; break;  // Spade
            case ('C'): type=5; break;  // Club
            case ('D'): type=4; break;  //diamonds
        }
        //pour les couleurs
        if(pt->C->couleur == 'R'){
            Color(4,15);
            if (pt->C->chiffre == 1) printf("A %c \t", type);
            else if (pt->C->chiffre == 11) printf("J %c \t",typ
            else if (pt->C->chiffre == 12) printf("Q %c \t",typ
            else if (pt->C->chiffre == 13) printf("K %c \t",typ
            else printf("%d %c \t", pt->C->chiffre, type);
            Color(15,0);
        }
```

Ensuite il faut demander à l'utilisateur quelle opération il veut effectuer et tout cela se répète dans une boucle while qui se termine soit par son gain à travers la fonction '**GagnerJeu'** ou bien sa perte à travers la fonction **'Blocage'**

```
while(GagnerJeu(Zone3) == 0 && Blockage(Zone1,Zone2,Zone3)==0){//boucle while termine quand l'utilisateur perd ou gagne la partie
```

Fonction GagnerJeu : fonction qui retourne 1 (l'utilisateur a gagné) si toutes les piles de la zone 3 se remplissent (c'est-à-dire CountPile de toutes les piles devra égale à 13) et retourne 0 sinon.

```
int GagnerJeu(Pile **P){
    int i;
    int count=0;
    for(i=0;i<4;i++){
        if(CountPile(P[i])==13)
            count++;
    }
    if(count==4)
        return 1;
    return 0;
}
```

Fonction Blocage : cette fonction retourne 0 (l'utilisateur n'est pas bloqué) si une pile de la zone 1 ou 2 est nulle sinon, elle voit les head (cartes) des piles des 2 zones s'il y a un déplacement qui peut avoir lieu entre eux ou entre eux et les piles de zone 3. S'il n'y a aucun déplacement qui peut avoir lieu elle retourne 1 (l'utilisateur se bloque = perd) sinon elle retourne 0.

```
    //fonction blockage principale
int Blockage(Pile **P1, Pile **P2, Pile **P3){
    int i,j,k,l,m=0,count=0;
    for(i=0;i<8;i++){
        if(P1[i]==NULL)
            return 0;
    }
    for(i=0;i<4;i++){
        if(P2[i]==NULL)
            return 0;
    }
    Card *C1[8];
    Card *C2[4];
    Card *C3[4];
    j=0;
    for(i=0;i<8;i++){
        C1[i]=peek(P1[j]);
        j++;
    }
    j=0;
    for(i=0;i<4;i++){
        C2[i]=peek(P2[j]);
        j++;
    }
```

Figure 1 : fonction blocage partie 1

```
if(P3[0]==NULL){
    C3[0]=initialiserCard(0,'C','N');
}
else
    C3[0]=peek(P3[0]);
if(P3[1]==NULL){
    C3[1]=initialiserCard(0,'H','R');
}
else
    C3[1]=peek(P3[1]);
if(P3[2]==NULL){
    C3[2]=initialiserCard(0,'S','N');
}
else
    C3[2]=peek(P3[2]);
if(P3[3]==NULL){
    C3[3]=initialiserCard(0,'D','R');
}
else
    C3[3]=peek(P3[3]);
```

Figure 2: Fonction blocage partie 2

```
    for(i=0;i<8;i++){
        for(j=i+1;j<8;j++){
            if(BlockageZone1(C1[i],C1[j])==1)
                count++;
        }
        for(k=0;k<4;k++){
            if(BlockageZone1_Zone2(C1[i],C2[k])==1)
                count++;
        }
        for(l=0;l<4;l++){
            if(BlockageZone3(C1[i],C3[l])==1)
                count++;
        }
    }
    for(i=0;i<4;i++){
        for(l=0;l<4;l++){
            if(BlockageZone3(C2[i],C3[l])==1)
                count++;
        }
    }
    if(count==108)
        return 1;
    return 0;
}
```

Figure 3: fonction blocage partie 3

Remarque : if(count==108) parce que le nombre maximale des fois dont on peut entrer dans les conditions if(Blockage…) est 108.

On remarque que la fonction blocage se compose de 3 fonctions : ‘**BlocageZone3’, ‘BlocageZone1’, ‘BlocageZone1-Zone2.**

BlocageZone1 : vérifie les déplacements entre les head des piles de la zone1, elle retourne 1 sir les 2 cartes passés en paramètres, la première ne peut pas être sous la deuxième et vice versa

```
int BlocageZone1(Card *c1 , Card *c2){
    if((c2->chiffre-c1->chiffre == 1 && c2->couleur!=c1->couleur) || (c1->chiffre-c2->chiffre == 1 && c2->couleur!=c1->couleur)){
        return 0;
    }
    return 1;
}
```

BlocageZone1_Zone2 : vérifie les déplacements entre les head des piles de la zone2 avec ceux des head des piles de la zone 1, elle retourne 1 sir les 2 cartes passés en paramètres, la première ne peut pas être sous la deuxième.

```
int BlocageZone1_Zone2(Card *c1,Card *c2){
    if (c1->chiffre-c2->chiffre == 1 && c2->couleur!=c1->couleur){
        return 0;
    }
    return 1;
}
```

BlocageZone3 : Vérifie les déplacements entre les head des piles des zone 1 ou 2 avec les piles de zone 3. Retourne 1 si les 2 cartes passés en paramètre la première ne peut pas être sous la deuxième.

```c
int BlocageZone3(Card *c1 , Card *c2){
    if (c1->type == c2->type && c1->chiffre-c2->chiffre == 1){
        return 0;
    }
    return 1;
}
```

Opérations :

D'abord pour demander à l'utilisateur quelle opération il veut effectuer on utilise la fonction Jeu qui demande à l'utilisateur d'entrer un nombre valide.

```c
do{
    temp=Jeu(&jeu);
}while(temp==0);
```

```c
    //pour demader à l'utilisateur qu'est-ce-qu'il veut faire
int Jeu(int *jeu){
    char buffer[256];
    do{
        printf("\t\t veuillez tapez un nombre entre 1 et 6 : ");
        if(scanf("%d",jeu)!=1){
            Color(4,0);
            printf("\t\t Errreur de saisie . Entrez un entier \n");
            Color(15,0);
            fgets(buffer, sizeof(buffer), stdin);
            *jeu=0;
        }
    }while(*jeu==0);
    if(*jeu > 6 || *jeu < 1){
        Color(4,0);
        printf("\t\t Erreur . veuillez tapez un nombre convenable s'il vous plait \n");
        Color(15,0);
        return 0;
    }
    return 1;
}
```

Remarque : Le tableau buffer[256] et fgets(buffer, sizeof(buffer),stdin) et tout cela est si on ne sait jamais, l'utilisateur se trompe et il entre un type char à la place d'un entier parce que sinon le jeu va se bloquer.

Pour les opérations. J'aurai besoin de la fonction '**FromCharToPile'** parce que à chaque fois l'utilisateur va entrer le nom des piles ça sera un type 'char' et moi j'aurais besoin de manipuler les piles. Cette fonction prend en paramètre 'le char' que l'utilisateur va entrez et retourne la pile. (P1[0] = A, P1[1] = B ; …… ; P2[0] = I, P2[1] = J ;…… ;P3[0] = M, P3[1] = N…)

```c
    //pour transformer le caractere que l'utilisateur va entrer en pile qu'on peut manipuler
Pile *FromCharToPile(char C,Pile **P1, Pile **P2, Pile **P3){
    Pile *pi;
    int i,j=C;
    for(i=0;i<8;i++){
        if(j==i+65)
            pi=P1[i];
    }
    for(i=0;i<4;i++){
        if(j==i+73)
            pi=P2[i];
    }
    for(i=0;i<4;i++){
        if(j==i+77)
            pi=P3[i];
    }
    return pi;
}
```

Opération 1 : Si l'utilisateur tape sur 1. L'opération 1 aura lieu ; cette opération qui permet à l'utilisateur de déplacer une carte de la zone 2 ou 1 (il lui suffit de préciser le nom de la pile) dans une pile de la zone 1. Pour cela on va utiliser la fonction '**DeplacementToZone1'** qui ne retourne 1 que si le déplacement est valide c'est-à-dire la carte de la pile de début soit inférieure à celle de fin par la valeur 1 et qu'elles soient de couleurs différentes. Pour tout autre erreur elle retourne une autre valeur.

```
if(jeu==1)
    temp=DeplacementToZone1(&deb,&fi,&DEB,&FIN,Zone1,Zone2,Zone3);
```

```
    //fonction pour tout deplacement entre une zone1-2 et zone 1
int DeplacementToZone1(char *deb, char*fi, Pile **debut, Pile **fin, Pile **P1, Pile **P2, Pile **P3){
    printf("\t\t dans quelle pile se trouve cette carte (pile de debut soit de zone 1 ou de zone 2) : ");
    scanf(" %c", deb);
    printf("\t\t ou vous voulez la transferer (pile fin): ");
    scanf(" %c", fi);
    if (*deb > 'L' || *deb < 'A' || *fi > 'H' || *fi < 'A')
        return 0;
    *debut=FromCharToPile(*deb,P1,P2,P3);
    *fin=FromCharToPile(*fi,P1,P2,P3);
    if(*debut == NULL)
        return -1;
    Card *c1=peek(*debut);
    if(*fin == NULL){
        empiler(fin,c1);
        pop(debut);
        return 1;
    }
    Card *c2=peek(*fin);
    if (c2->chiffre-c1->chiffre!=1 || c2->couleur==c1->couleur)
        return -1;
    empiler(fin,c1);
    pop(debut);
    return 1;
}
```

Opération 2 : Si l'utilisateur tape sur 2. L'opération 2 aura lieu ; cette opération qui permet à l'utilisateur de déplacer une carte de la zone 2 ou 1 (il lui suffit de préciser le nom de la pile) dans une pile de la zone 2. Pour cela on va utiliser la fonction '**DeplacementToZone2'** qui ne retourne 1 que si le déplacement est valide c'est-à-dire la pile de fin qui est une pile de la zone 2 soit égale à NULL. Pour tout autre erreur elle retourne une autre valeur.

```
else if(jeu==2)
    temp=DeplacementToZone2(&deb,&fi,&DEB,&FIN,Zone1,Zone2,Zone3);
```

```
    //fonction pour tout deplacement entre une zone1-2 et zone 2
int DeplacementToZone2(char *deb, char*fi, Pile **debut, Pile **fin, Pile **P1, Pile **P2, Pile **P3){
    printf("\t\t dans quelle pile se trouve cette carte (pile de debut soit de zone 1 ou de zone 2) : ");
    scanf(" %c", deb);
    printf("\t\t ou vous voulez la transferer (pile de fin) : ");
    scanf(" %c", fi);
    if (*deb >'L' || *deb < 'A' || *fi > 'L' || *fi < 'I')
        return 0;
    *debut=FromCharToPile(*deb,P1,P2,P3);
    *fin=FromCharToPile(*fi,P1,P2,P3);
    if (*fin != NULL || *debut == NULL)
        return -1;
    Card *c1=peek(*debut);
    empiler(fin,c1);
    pop(debut);
    return 1;
}
```

Opération 3 : Si l'utilisateur tape sur 3. L'opération 3 aura lieu ; cette opération qui permet à l'utilisateur de déplacer une carte de la zone 2 ou 1 (il lui suffit de préciser le nom de la pile) dans une pile de la zone 3. Pour cela on va utiliser la fonction '**DeplacementToZone3**' qui ne retourne 1 que si le déplacement est valide c'est-à-dire si la pile de fin qui est une pile de la zone 3 est NULL ; Il faut que la carte de la pile de début soit le 'AS' (en respectant le type de la pile bien sûr), sinon il faut que la carte de la pile de fin soit inférieur à celle de début par la valeur 1 et qu'elles soient du même type. Pour tout autre erreur elle retourne une autre valeur.

```
else if(jeu==3)
    temp=DeplacementToZone3(&deb,&fi,&DEB,&FIN,Zone1,Zone2,Zone3);
```

```
    //fonction pour tout deplacement entre une zone1-2 et zone 3
int DeplacementToZone3(char *deb, char*fi, Pile **debut, Pile **fin, Pile **P1, Pile **P2, Pile **P3){
    printf("\t\t dans quelle pile se trouve cette carte (pile de debut soit de zone 1 ou de zone 2) : ");
    scanf(" %c", deb);
    printf("\t\t ou vous voulez la transferer (pile de fin) : ");
    scanf(" %c", fi);
    if (*deb > 'L' || *deb < 'A' || *fi > 'P' || *fi < 'M')
        return 0;
    *debut=FromCharToPile(*deb,P1,P2,P3);
    *fin=FromCharToPile(*fi,P1,P2,P3);
    if(*debut == NULL)
        return -1;
    Card *c1=peek(*debut);
    if(*fin == NULL && c1->chiffre != 1 )
        return -1;
    if(*fin == NULL && c1->chiffre == 1){
        switch(*fi){ //pour respecter le type des piles quand les piles sont encores nulles
            case 'M':
                if(c1->type != 'C')
                    return -2;
                break;
            case 'N':
                if(c1->type != 'H')
                    return -2;
                break;
```

Figure : Déplacement To Zone 3 part 1

```
                if(c1->type != 'H')
                    return -2;
                break;
            case 'O':
                if(c1->type != 'S')
                    return -2;
                break;
            case 'P':
                if(c1->type != 'D')
                    return -2;
                break;
        }
        empiler(fin,c1);
        pop(debut);
        return 1;
    }
    if(*fin != NULL){
        Card *c2=peek(*fin);
            if (c1->chiffre-c2->chiffre!=1 || c2->type!=c1->type)
                return -1;
        empiler(fin,c1);
        pop(debut);
        return 1;
    }
}
```

Figure: Déplacement To Zone 3 part 2

On aura besoin de la variable de type entier ‘**temp’** qui va prendre comme valeur ce que les fonctions des déplacements retournent pour qu’on puisse savoir si le déplacement est réussi ou non et pour nous aider à afficher les erreurs

```
if(temp==0){
    printf("\t\t EREEUR veuillez choisir des piles convenables s'il vous plait \n");
}
else if(temp == -1){
    printf("\t\t ERREUR DE DEPLACEMENT \n");
}
else if (temp == -2){
    printf("\t\t Erreur : veuillez respectez le type de la pile \n");
}
```

Après chaque déplacement réussi on doit faire une mise à jour à nos piles pour un bon affichage c’est pour cela qu’on doit faire appel à la fonction ‘**UpdatePile**’ à la fin de la boucle.

```
void UpdatePile(char deb, char fi, Pile *debut, Pile *fin, Pile **P1, Pile **P2, Pile **P3){
    int i;
    int j=deb;
    int k=fi;
    for(i=0;i<8;i++){
        if(j==i+65)
            P1[i]=debut;
        if(k==i+65)
            P1[i]=fin;
    }
    for(i=0;i<4;i++){
        if(j==i+73)
            P2[i]=debut;
        if(k==i+73)
            P2[i]=fin;
    }
    for(i=0;i<4;i++){
        if(j==i+77)
            P3[i]=debut;
        if(k==i+77)
            P3[i]=fin;
    }
}
```

```
if(temp == 1)
    UpdatePile(deb,fi,DEB,FIN,Zone1,Zone2,Zone3);
```

Opération 4, 5, 6 : Ces opérations aident à quitter la partie en sortant de la boucle **« while(GagnerJeu(Zone3) == 0 && Blocage(Zone1,Zone2,Zone3)==0) »**

```
else if(jeu==4 || jeu==5 || jeu==6)
    break;
```

La différence c’est que si l’utilisateur tape sur 4 il va rejouer la même partie du début, et s’il tape sur 5 il va jouer partie différente, et s’il tape sur 6 il va quitter le jeu

complétement. Pour cela j'ai utilisé une boucle while infinie et j'ai mis tous le jeu (l'empilation des piles, l'affichage…..) dans cette boucle

```
while(1){ //boucle while infini pour la nouvelle partie
    //empiler les piles de la zone 1 aléatoirement
    for (i = 0; i < 7; i++) {//empiler 7 cartes dans les 4 p
        empiler(&Zone1[0], cartes[i]);
        empiler(&Zone1[1], cartes[i + 7]);
```

La boucle ne se termine que si l'utilisateur tape sur 6 (quitter complétement le jeu) ; et s'il tape sur 4 les cartes ne vont pas se mélanger une autre fois pour qu'il puisse rejouer la même partie, mais s'il tape sur 5 ils vont se mélanger pour que l'utilisateur rejoue une différente partie ; et bien sur toutes les piles doivent se réinitialiser à NULL.

```
PileNULL(Zone1,8); PileNULL(Zone2,4); PileNULL(Zone3,4);
if(jeu==5 || rejouer == 1){
    melangerCartes(cartes, tailleCartes);
}
else if(jeu==6 || rejouer==0){
    break;
}
```

VI. Dernière étape :

Si l'utilisateur gagne, la fonction '**GagnerJeu'** retournera 1 et la boucle « while(GagnerJeu(Zone3) == 0 && Blocage(Zone1,Zone2,Zone3)==0) » terminera. Et on va Féliciter l'utilisateur puis lui demander s'il veut rejouer ou bien quitter la partie en utilisant la fonction 'Rejouer'. Même principe quand il perd.

```
int rejouer;
if(GagnerJeu(Zone3) == 1) {
    Distribuer(Zone1,Zone2,Zone3);
    Color(2,0);
    printf("Felicitation . Vous avez gagne :) \n");
    Color(15,0);
    Rejouer(&rejouer);
}
if(Blocage(Zone1,Zone2,Zone3)==1){
    Distribuer(Zone1,Zone2,Zone3);
    Color(4,0);
    printf("Desole mais Vous avez perdu :( \n");
    Color(15,0);
    Rejouer(&rejouer);
}
PileNULL(Zone1,8); PileNULL(Zone2,4); PileNULL(Zone3,4);
if(jeu==5 || rejouer == 1){
    melangerCartes(cartes, tailleCartes);
}
else if(jeu==6 || rejouer==0){
    break;
}
```

```c
void Rejouer(int *rejouer){
    printf("voulez vous continuer a jouer ? \n");
    printf("Pour rejouez cette partie veuillez tapez sur 2\n");
    printf("Pour une nouvelle partie veuillez tapez sur 1\n");
    printf("Pour quitter veuillez tapez sur 0\n");
    do{
    printf("tapez un nombre entre 0 et 2 : ");
    scanf("%d",rejouer);
    Color(4,0);
    if(*rejouer < 0 || *rejouer > 2)
        printf("Erreur . veuillez tapez un nombre convenable s'il vous plait \n");
    Color(15,0);
    }while(*rejouer < 0 || *rejouer > 2);
}
```

VII. Design et Affichage :

Couleur : Pour les couleurs j'ai utilisé bien sur la bibliothèque **« windows.h »** et la fonction '**Color'** qu'il suffit de l'appeler avant ce qu'on veut afficher et de lui passer en paramètre la couleur du texte et la couleur du fond

```c
void Color(int couleurDuTexte,int couleurDeFond) // fonction d'affichage de couleurs
{
        HANDLE H=GetStdHandle(STD_OUTPUT_HANDLE);
        SetConsoleTextAttribute(H,couleurDeFond*16+couleurDuTexte);
}
```

Symboles : Pour les symboles il suffit de déclarer un type 'char' et lui donner un nombre de 3 à 6 ça dépend du symbole.

```c
char type; //pour transformer le type qui est un caractere en symbole
    switch (pt->C->type) {
        case ('H'): type=3; break;  // Heart
        case ('S'): type=6; break;  // Spade
        case ('C'): type=5; break;  // Club
        case ('D'): type=4; break;  //diamonds
    }
```

Effacer l'écran : Pour l'affichage à l'écran on doit à chaque itération de la boucle d'effacer ce qui précède. Pour cela on doit utiliser les deux fonctions : **'effacerEcranWindows'** et **'effacerEcranUnix'**

```
void effacerEcranUnix() {
    system("clear");
}

// Fonction pour effacer l'écran (Windows)
void effacerEcranWindows() {
    system("cls");
}
```

```
#ifdef _WIN32
    effacerEcranWindows();
#else
    effacerEcranUnix();
#endif
```

Fin du Rapport

Merci pour votre attention

Boulghalegh Youssef